

File Management

Adiesha Liyana Ralalage

File and File systems

- Data collections created by users.
- The File System is one of the most important parts of the OS to a user.
- Desirable properties of files:

Long-term existence

- Files are stored on disk or other secondary storage and do not disappear when a user logs off

Sharable between processes

- Files have names and can have associated access permissions that permit controlled sharing

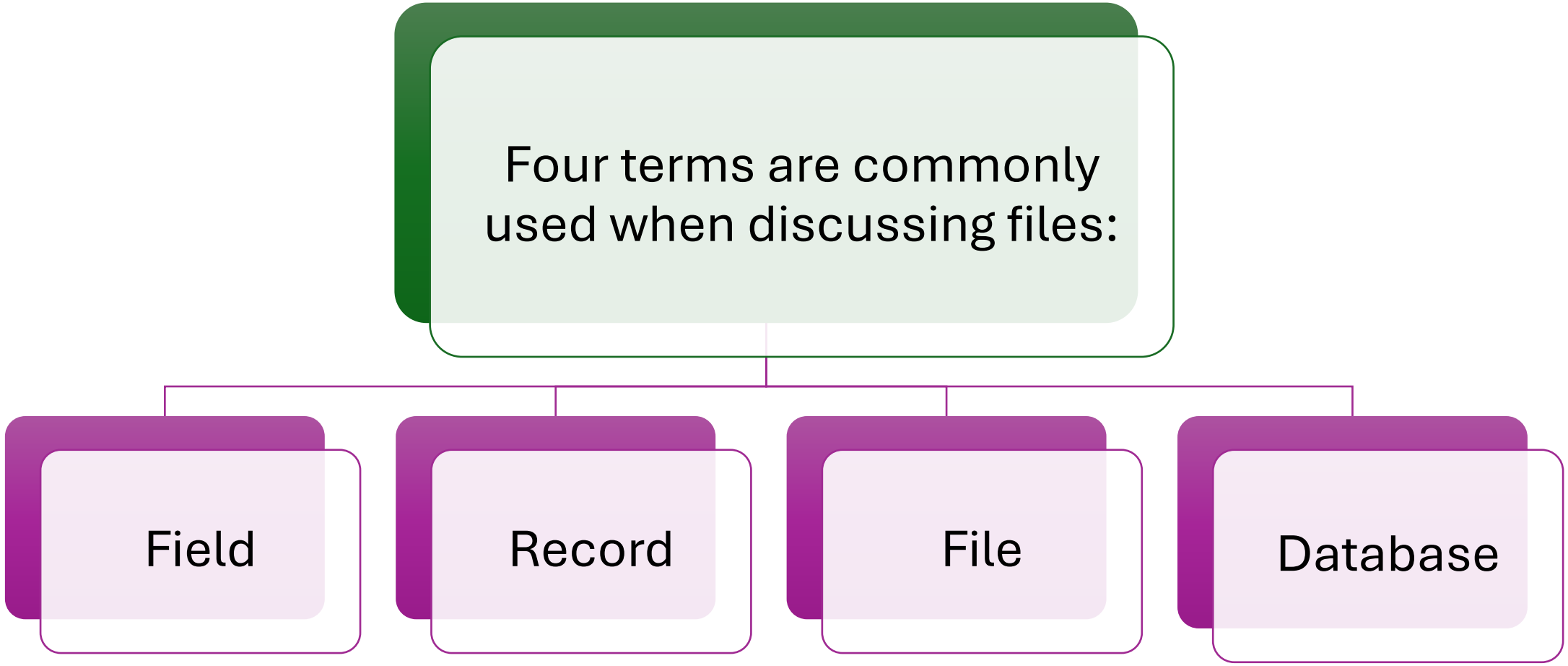
Structure

- Files can be organized into hierarchical or more complex structure to reflect the relationships among files

File system

- Provide a means to store data organized as files as well as a collection of functions that can be performed on files.
- Maintain a set of attributes associated with the file.
  - Owner, creation time, last modified date, etc.
- Typical operations include:
  - Create
  - Delete
  - Open
  - Close
  - Read
  - Write

Conceptual model of data organization



- Files may contain structured information organized into fields and records, and databases may use files to store their data. However, modern general-purpose operating systems typically treat regular files as sequences of bytes, leaving the interpretation of their contents to applications.

Terms

Field

- Basic element of data.
- Contains a single value.
- Fixed or variable length.

Record

- Collection of related fields that can be treated as a unit by some application program
- Fixed or variable length.

Database

- Collection of related data
- Relationships among elements of data are explicit
- Designed for use by a number of different applications
- Consists of one or more types of files

File

- Collection of similar records
- Treated as a single entity
- May be referenced by name
- Access control restrictions usually apply at the file level

- It should be noted that not all file systems exhibit the sort of structure discussed
- On UNIX and UNIX-like systems, the basic file structure is just a stream of bytes.
- For example, a C program is stored as a file but does not have physical fields, records, and so on.

OS Type	File View	Example Systems
Byte-oriented	File = sequence of bytes	UNIX, LINUX, Windows
Record-Oriented	File=sequence of records	IBM OS/360, MVS, z/OS, OpenVMS, older DEC systems

File Management System objectives

- Meet the data management needs of the user.
- Guarantee that the data in the file are valid.
- Optimize performance.
- Provide I/O support for a variety of storage device types.
- Minimize the potential for lost or destroyed data.
- Provide a standardized set of I/O interface routines to user processes.
- Provide I/O support for multiple users in the case of multiple-user systems.

Minimal user requirements

- Each user

1. Should be able to create, delete, read, write and modify files
2. May have controlled access to other users' files
3. May control what type of accesses are allowed to the users' files
4. Should be able to move data between files
5. Should be able to back up and recover files in case of damage
6. Should be able to access his or her files by name rather than by numeric identifier

File system architecture

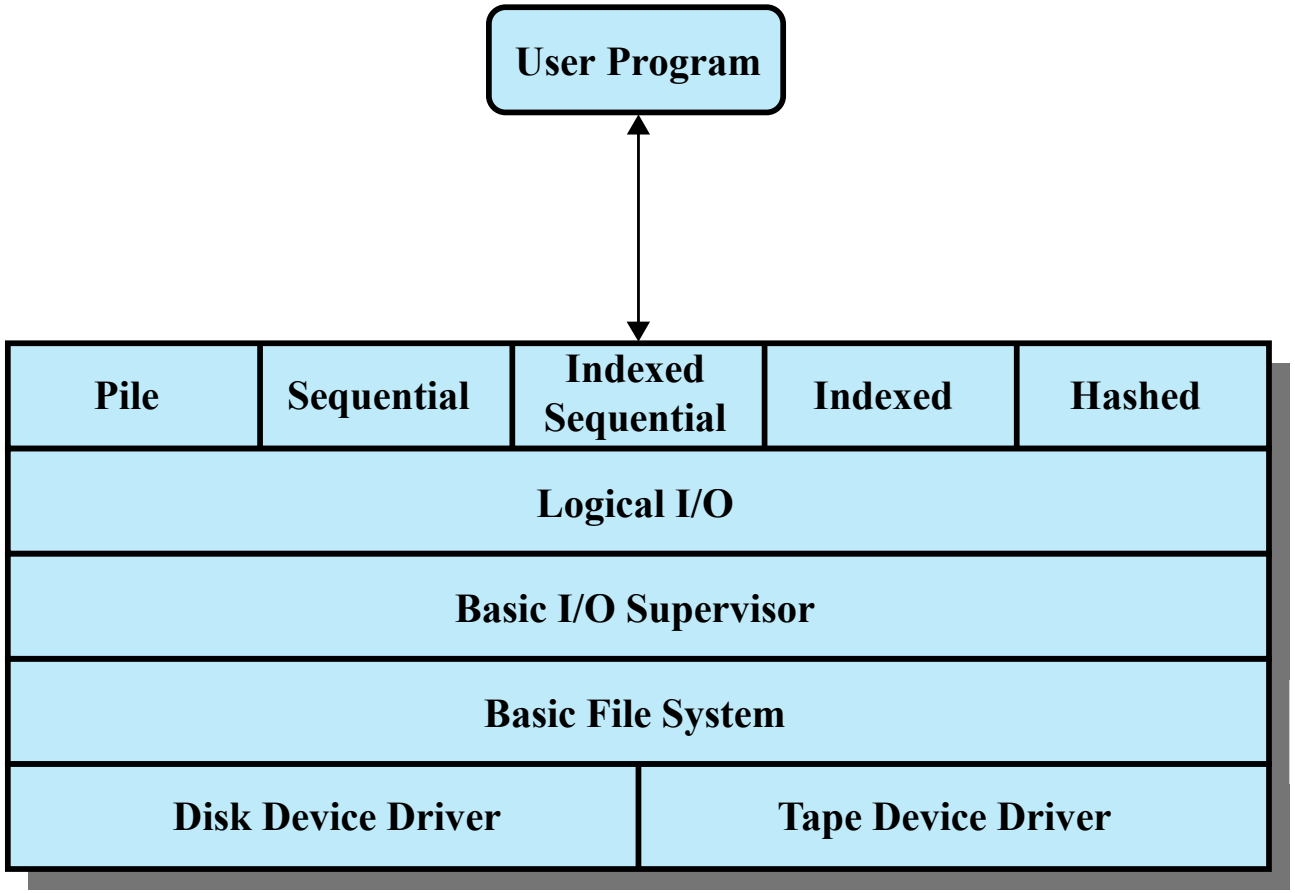


Figure 12.1 File System Software Architecture

File management functions

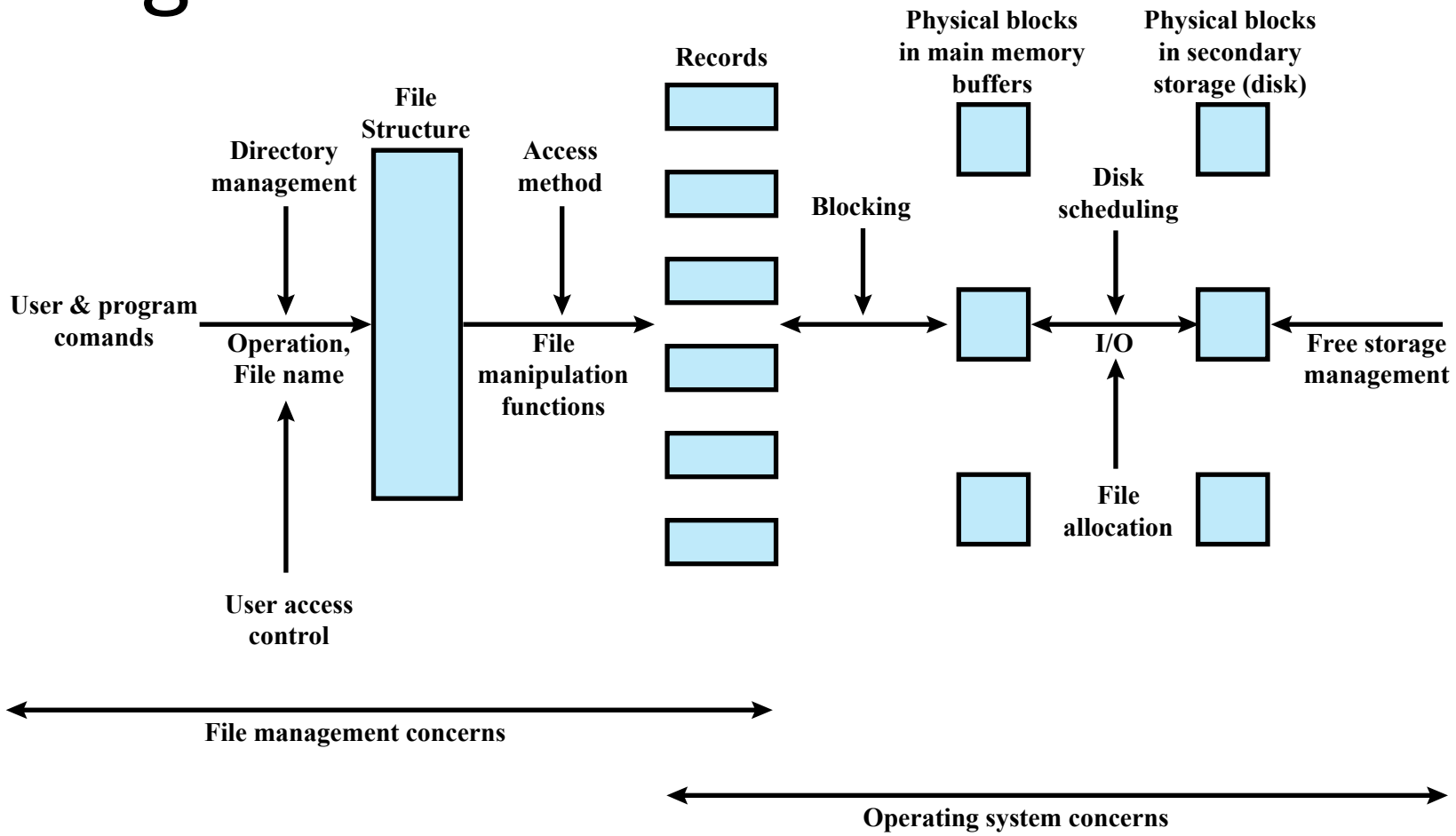
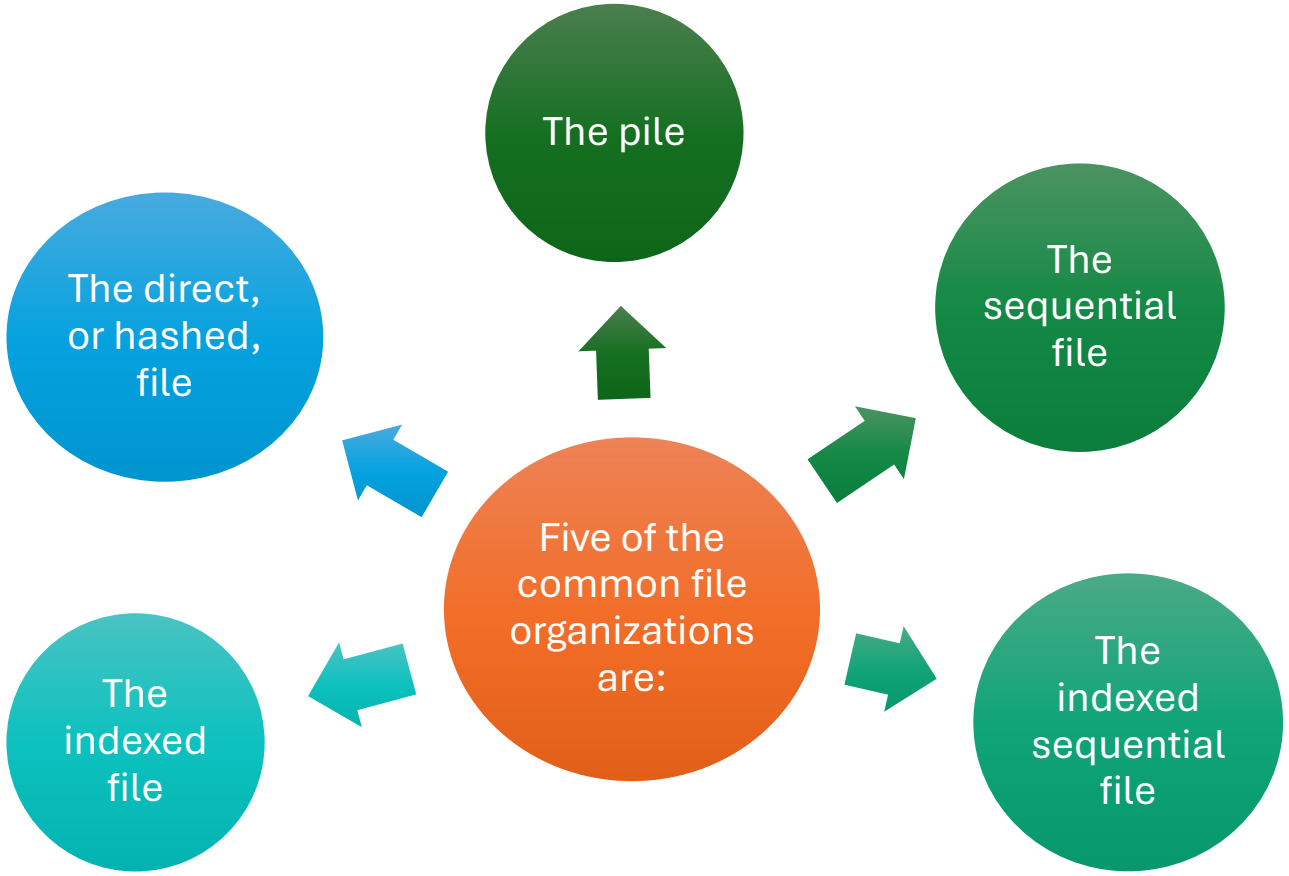


Figure 12.2 Elements of File Management

File Organization and Access

- File organization is the **logical structuring of the records as determined by the way in which they are accessed.**
- In choosing a file organization, several criteria are important:
  - Short access time
  - Ease of update
  - Economy of storage
  - Simple maintenance
  - Reliability
- Priority of criteria depends on the application that will use the file.

File organization types



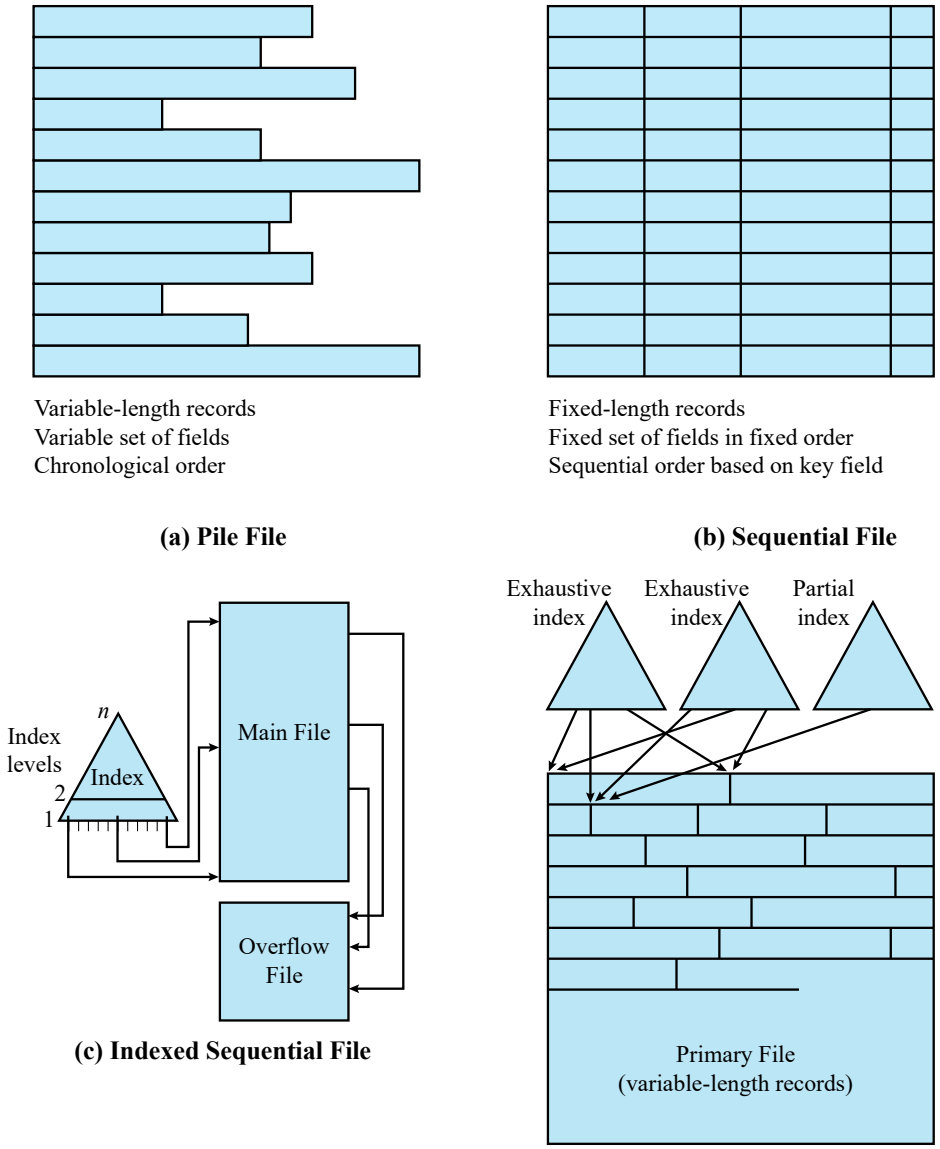
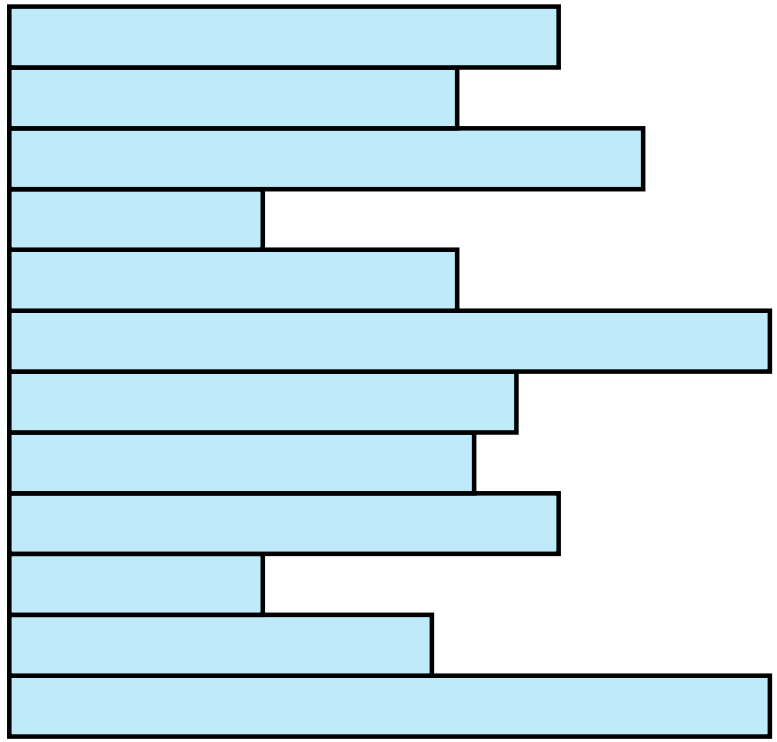


Figure 12.3 Common File Organizations

The Pile

- Least complicated form of file organization
- Data are collected in the order they arrive
- Each record consists of one burst of data
- Purpose is simply to accumulate the mass of data and save it
- Record access is by exhaustive search



Variable-length records
Variable set of fields
Chronological order

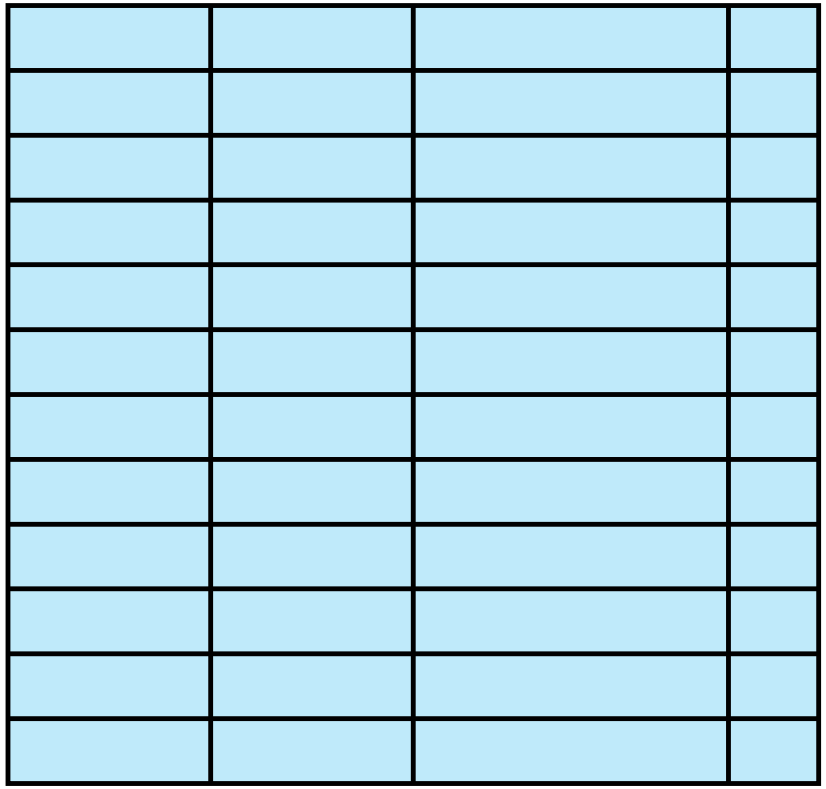
(a) Pile File

The Pile

- If we wish to find a record that contains a particular field with a particular value, it is necessary to examine each record in the pile until the desired record is found.
- If we want to find all the records that contain a particular field, then all the records should be searched.
- These types of files uses space well.
- Easy to append.

The Sequential File

- Records are stored in order of a designated key field.
- In the traditional model, records have a fixed format, and the key uniquely identifies each record.
- Well suited to batch processing, where many or all records are processed in sequence.
- Can be stored on disk or tape; sequential access is especially well suited to tape.
- Without an auxiliary index, finding a particular record generally requires a sequential search.



Fixed-length records
Fixed set of fields in fixed order
Sequential order based on key field

(b) Sequential File

The Sequential File

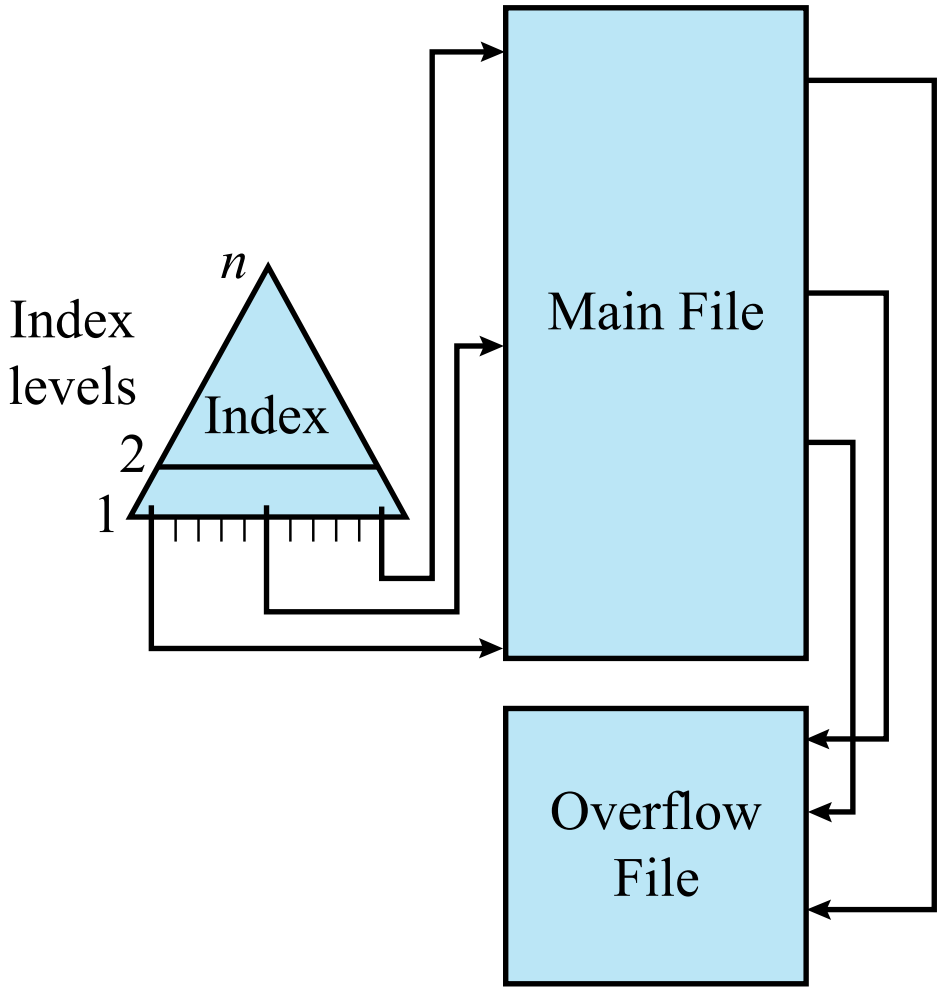
- Works very well if you must process all the records in the file.
- Typically, a sequential file is stored in simple sequential ordering of the records within blocks.
- That is, the physical organization of the file on tape or disk directly matches the logical organization of the file.
- For interactive applications that involve queries and/or updates of individual records, the sequential file provides poor performance.
  - Access requires the sequential search of the file for a key match.
  - If the entire file, or a large portion of the file, can be brought into main memory at one time, more efficient search techniques are possible.

The Sequential File

- Since the logical organization and physical organization matches in sequential files, when we try to insert a record, it becomes harder to update in the secondary storage.
- Typically, a separate **pile file** called a log file, or transaction file is maintained for inserting new records.
- **The periodically batch update is performed to merge the separate file in the master file to produce correct key sequence.**

Indexed Sequential File

- Adds an index to the file to support random access.
- Adds an overflow file
- Greatly reduces the time required to access a single record
- Multiple levels of indexing can be used to provide greater efficiency in access



(c) Indexed Sequential File

Indexed sequential file

- Index is a simply a simple sequential file.
- Each record in the index file consists two things.
  - Key field
  - Pointer into the main file.
- To find a specific field value, index file is searched sequentially to find the highest key value that is equal or precedes the desired key value.
- Then we read that pointer value and start searching the main file from that point onwards.

Indexed sequential file

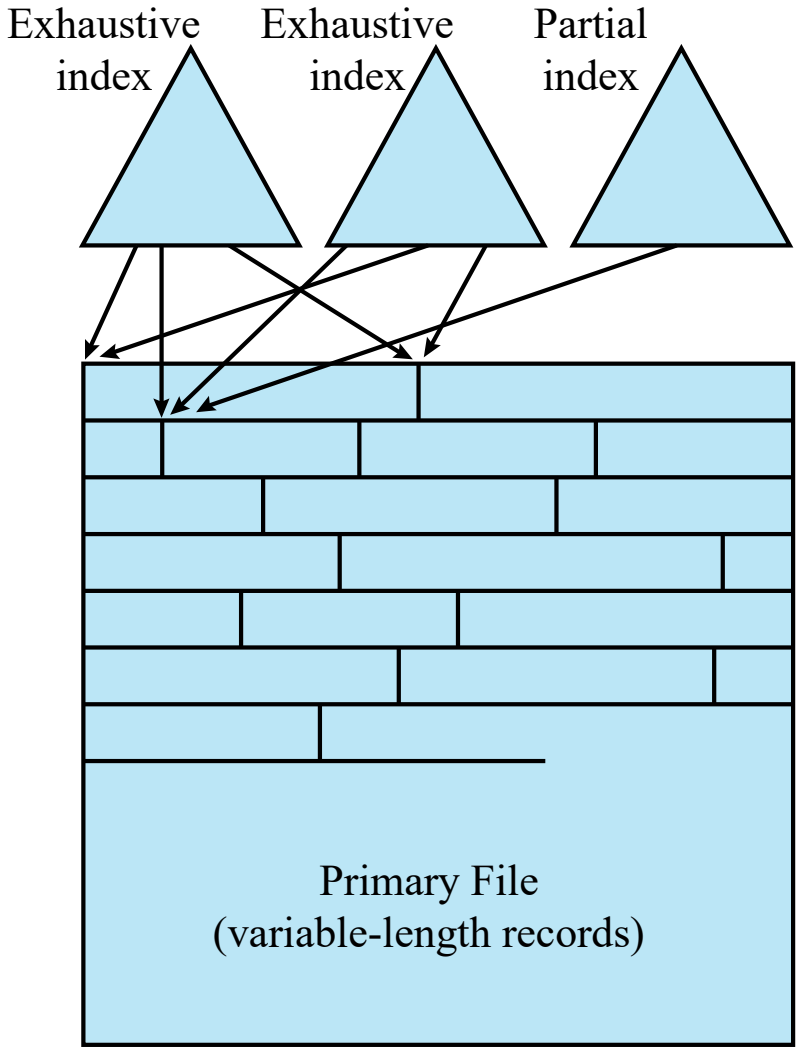
- Suppose you have a sequential file with 1 million records.
- To search for a particular key, on average, it will require half a million record accesses.
- Now suppose you have an index with 1000 entries, with keys in the index evenly distributed over the main file.
- Now it will take on average 500 accesses to search over the index and 500 accesses to main file to find the record.
- Average search length is reduced from 500000 to 1000.

Indexed sequential file

- The indexed sequential file greatly reduces the time required to access a single record, without sacrificing the sequential nature of the file.
- We can even add multiple levels of indexing for greater efficiency.

Indexed File

- Records are accessed only through their indexes.
- Variable-length records can be employed.
- In the general indexed file, the concept of sequentiality and single key is abandoned.
- Exhaustive index contains one entry for every record in the main file
- Partial index contains entries to records where the field of interest exists
- Used mostly in applications where timeliness of information is critical
- Examples would be airline reservation systems and inventory control systems.
- The index itself is a sequential file.
- We need lot of additional metadata for indexed files.



(d) Indexed File

Direct or Hashed File

- Uses a hash function to map a record's key to a storage bucket.
- Provides fast retrieval when the exact key is known.
- Commonly used for frequent individual-record lookups.
- Can support fixed-length or variable-length records.
- A good hash function distributes records evenly across buckets, reducing collisions.
- Collisions can be handled using overflow buckets linked together through chaining.
- Less suitable for range queries or processing records in key order.
- Works on both HDDs and SSDs; SSDs reduce the cost of random storage access.

Examples are:

- Directories
- Pricing tables
- Schedules
- Name lists

Secondary Storage

- On secondary storage, a file consists of a collection of blocks.
- The operating system or file management system is responsible for allocating blocks to files.

Record Blocking

- Blocks are the unit of I/O with secondary storage
- For I/O to be performed records must be organized as blocks
- Given the size of a block, three methods of blocking can be used:

1. **Fixed-Length Blocking –** fixed-length records are used, and an integral number of records are stored in a block
    1. Internal fragmentation – unused space at the end of each block
2. **Variable-Length Spanned Blocking –** variable-length records are used and are packed into blocks with no unused space
3. **Variable-Length Unspanned Blocking –** variable-length records are used, but spanning is not employed

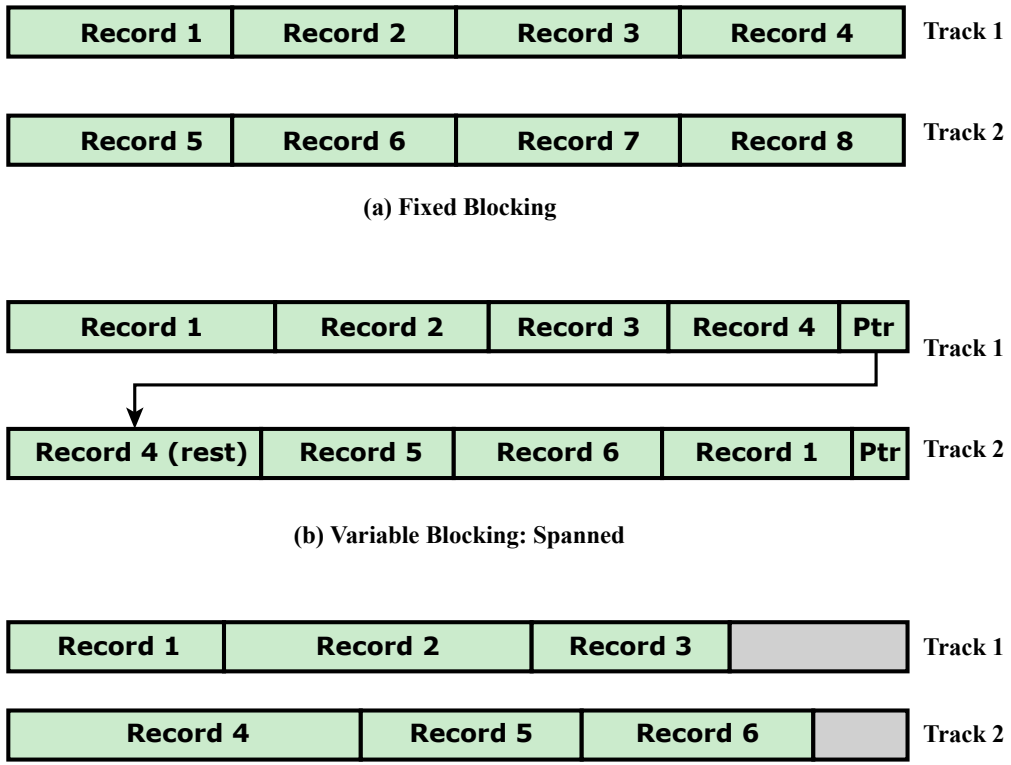


Figure 12.8 Record Blocking Methods

File Allocation

- On secondary storage, a file consists of a collection of blocks
- The operating system or file management system is responsible for allocating blocks to files
- The approach taken for file allocation may influence the approach taken for free space management.
- Space is allocated to a file as **one or more portions** (contiguous set of allocated blocks)

File Allocation

- Several issues needs to be considered for file allocation.
  - **Should we allocate the maximum required space when creating a file?**
  - **What size “portion” should be allocated?**
  - **How do we track which storage chunks belong to a file?**
    - The filesystem needs metadata that maps the file’s contents to storage locations.
    - Ex: File allocation table (FAT)
      - Data structure used to keep track of the portions assigned to a file.
      - Used in DOS operating system.

Preallocation vs Dynamic Allocation

- A preallocation policy requires that the maximum size of a file be declared at the time of the file creation request.
- For many applications it is difficult to estimate reliably the maximum potential size of the file
  - Tends to be wasteful because users and application programmers tend to overestimate size
- Dynamic allocation allocates space to a file in portions as needed

Portion size?

- The second issue listed is that of the size of the portion allocated to a file.
  - A portion large enough to hold the entire file is allocated.
  - we can allocate block by block to the file.
- We need something in between.
- If portion size is small, we must keep track of lot of information.
- If portion size is too large, we may be wasting space.

Too major approaches for portion size

Variable, large contiguous portions

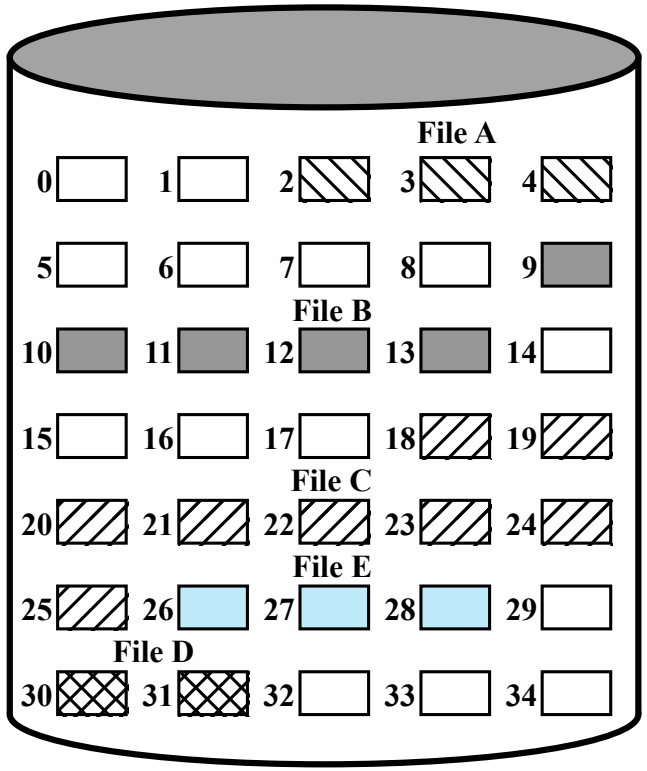
- Provides better performance
- The variable size avoids waste
- The file allocation tables are small
- Kind of like dynamic partitioning in memory management
  - Best-Fit
  - Next-Fit
  - Nearest-Fit

Small fixed potions (blocks)

- Small fixed portions provide greater flexibility
- They may require large tables or complex structures for their allocation
- Contiguity has been abandoned as a primary goal
- Blocks are allocated as needed

File allocation methods

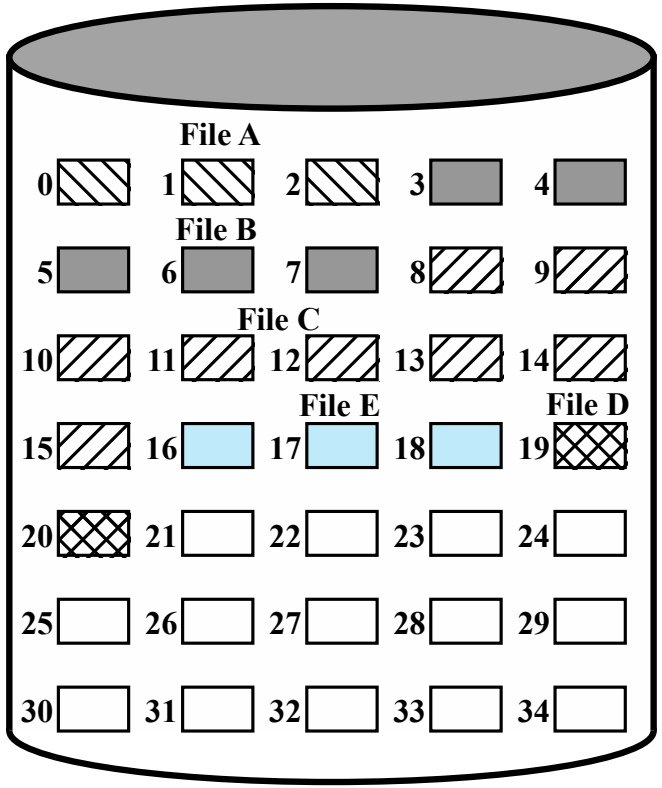
- **Contiguous**
  - Single contiguous set of blocks are allocated to the file at the time of creation.
  - Pre-allocation is necessary.
  - Large portion size.
  - Only one entry in the file allocation table for a file
- Chained
- Indexed



File Allocation Table

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

Figure 12.9 Contiguous File Allocation



File Allocation Table

File Name	Start Block	Length
File A	0	3
File B	3	5
File C	8	8
File D	19	2
File E	16	3

Figure 12.10 Contiguous File Allocation (After Compaction)

File allocation methods

- Contiguous
- **Chained**
  - Allocation is on an individual block basis.
  - Each block contain a pointer to the next block
  - Preallocation is possible but mostly allocate blocks as we need them.
  - Any free block can be allocated to the file.
  - No external fragmentation.
  - Best suitable for sequential files that are processed sequentially.
  - Only one entry for each file in File Allocation Table.
  - No principle of locality
  - Consolidation can be performed to bring blocks of files together.
- Indexed

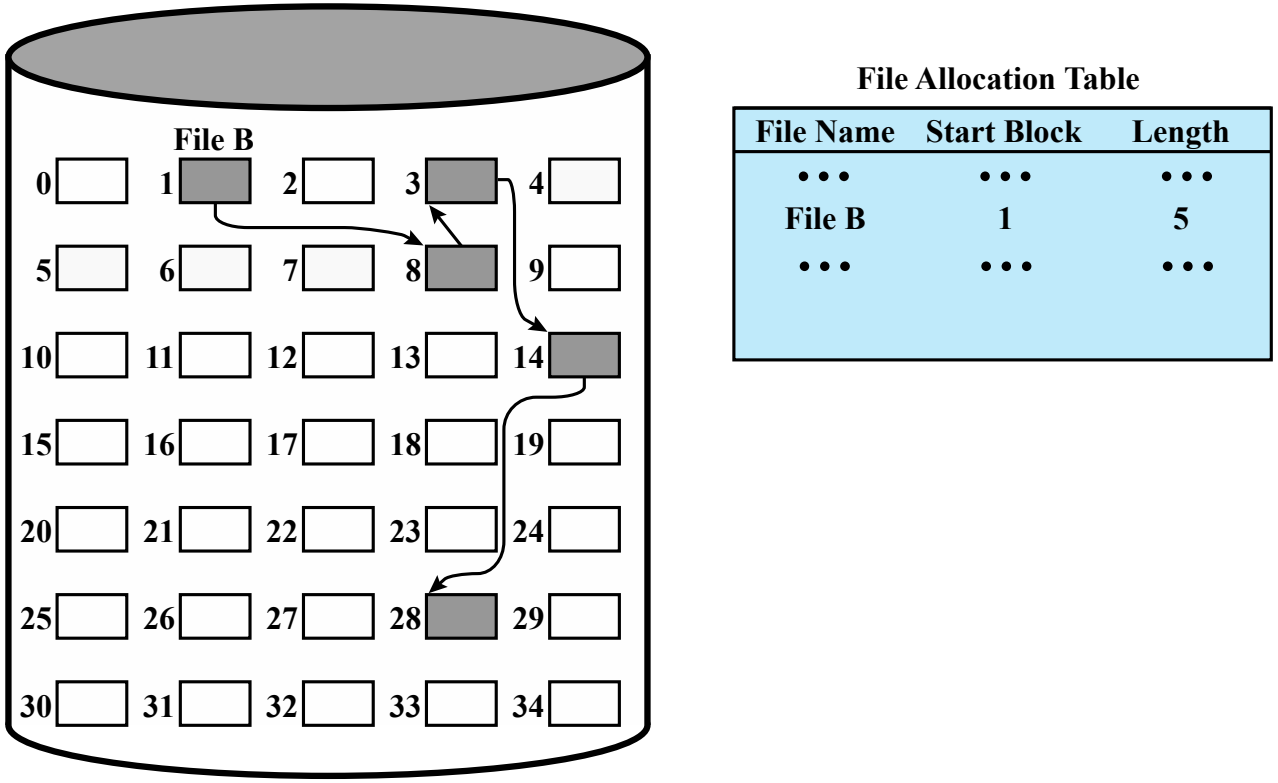


Figure 12.11 Chained Allocation

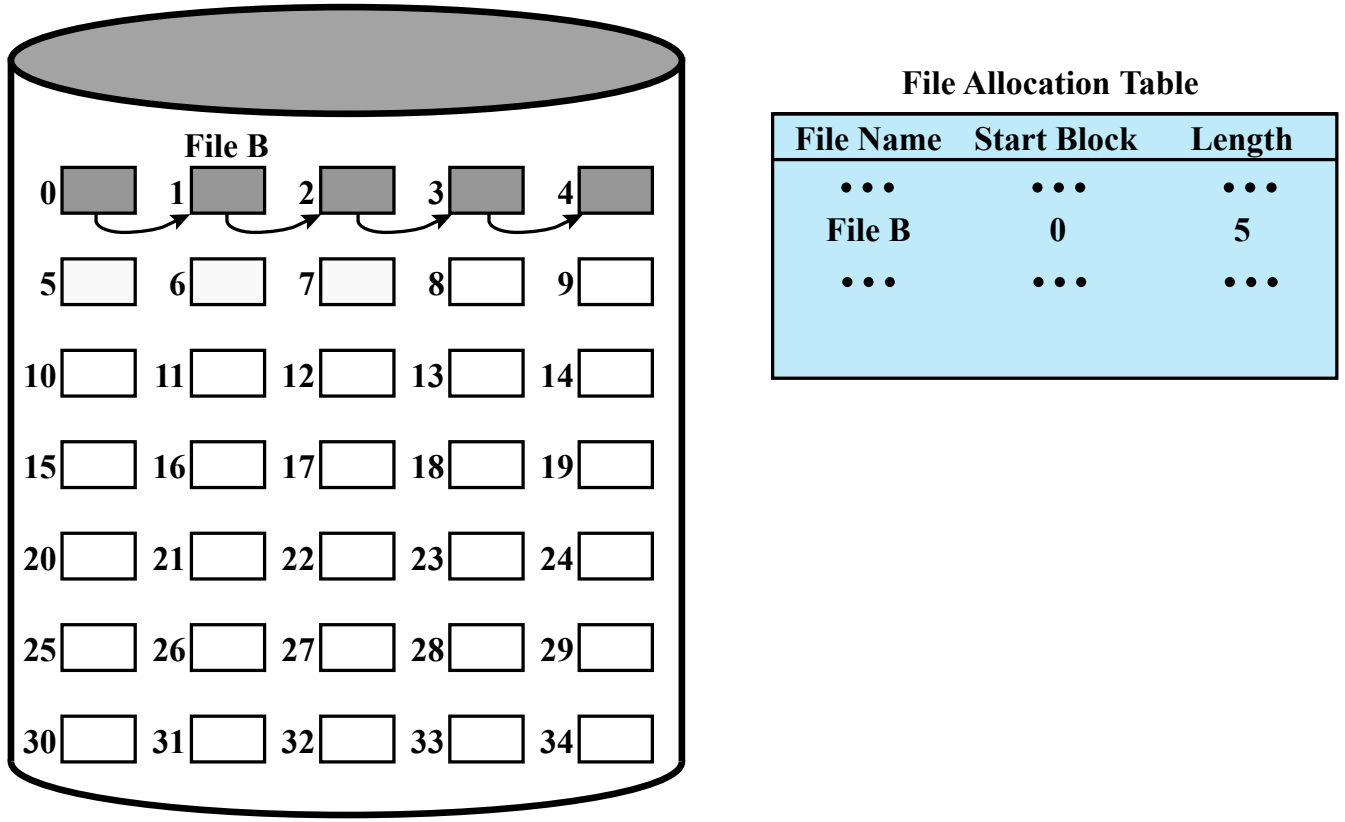


Figure 12.12 Chained Allocation (After Consolidation)

File allocation methods

- Contiguous
- Chained
- Indexed
  - One level index is kept for each file.
  - Each entry in the index keeps track of a particular portion.
  - Can use variable size or fixed size portions,
  - Index is kept at a separate block.
  - Variable size potions increase locality, while fixed size blocks remove external fragmentation.
  - Consolidation can be done time to time to improve locality.
  - Supports both sequential and direct access to the file.

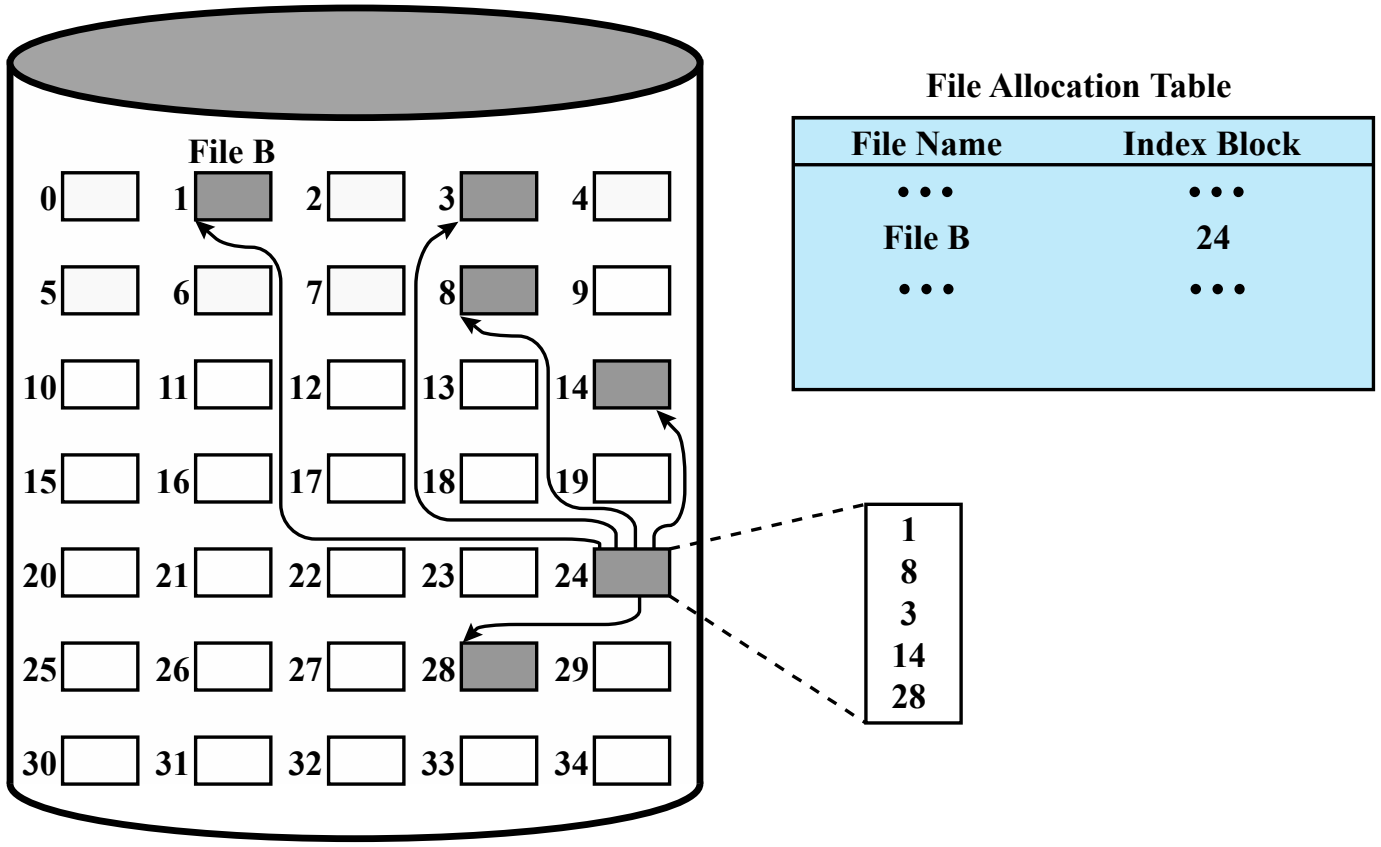


Figure 12.13 Indexed Allocation with Block Portions

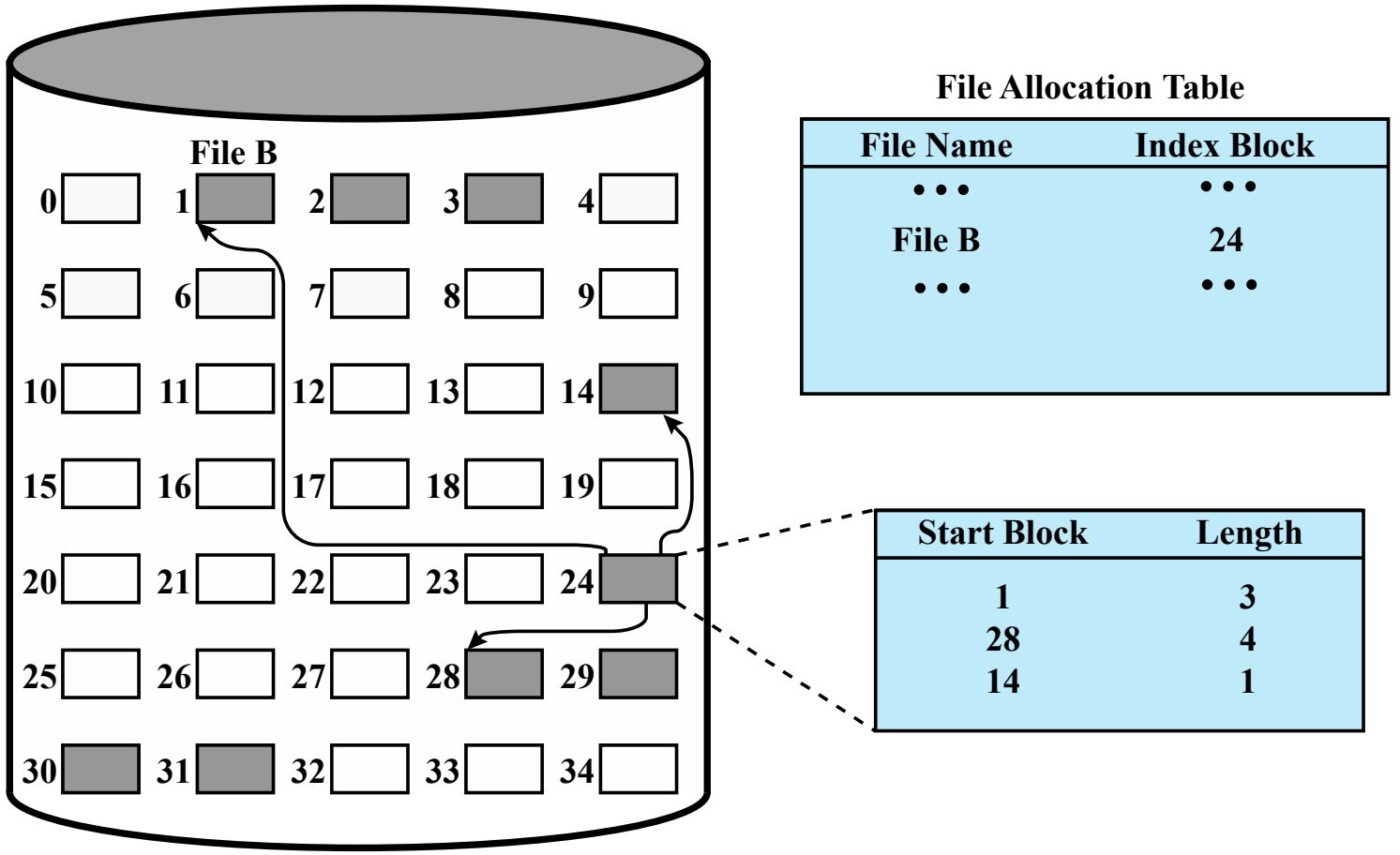


Figure 12.14 Indexed Allocation with Variable-Length Portions

File Allocation Methods

	Contiguous	Chained	Indexed	
Preallocation?	Necessary	Possible	Possible	
Fixed or variable size portions?	Variable	Fixed blocks	Fixed blocks	Variable
Portion size	Large	Small	Small	Medium
Allocation frequency	Once	Low to high	High	Low
Time to allocate	Medium	Long	Short	Medium
File allocation table size	One entry	One entry	Large	Medium

Free space management

- Just as allocated space must be managed, so must the unallocated space
- To perform file allocation, it is necessary to know which blocks are available
- A **disk allocation table** is needed in addition to a file allocation table

Bit tables

- This method uses a vector containing one bit for each block on the disk
- Each entry of a 0 corresponds to a free block, and each 1 corresponds to a block in use

Advantages:

- Works well with any file allocation method
- It is as small as possible

Chained Free Portions

- The free portions may be chained together by using a pointer and length value in each free portion
- Negligible space overhead because there is no need for a disk allocation table
- Suited to all file allocation methods

Disadvantages:

- Leads to fragmentation—After some use, the disk will become quite fragmented, and many portions will be a single block long.
- Every time you allocate a block you need to read the block first to recover the pointer to the new first free block before writing data to that block

Indexing

- Treats free space as a file and uses an index table as it would for file allocation.
- There is one entry in the table for every free portion on the disk.
- For efficiency, the index should be on the basis of variable-size portions rather than blocks
- This approach provides efficient support for all of the file allocation methods

Free Block List

Each block is assigned a number sequentially

- The list of the numbers of all free blocks is maintained in a reserved portion of the disk

Depending on the size of the disk, either 24 or 32 bits will be needed to store a single block number

- The size of the free block list is 24 or 32 times the size of the **corresponding bit table** and must be stored on disk.

There are two effective techniques for storing a small part of the free block list in main memory:

- The list can be treated as a push-down stack with the first few thousand elements of the stack kept in main memory
- The list can be treated as a FIFO queue, with a few thousand entries from both the head and the tail of the queue in main memory.